

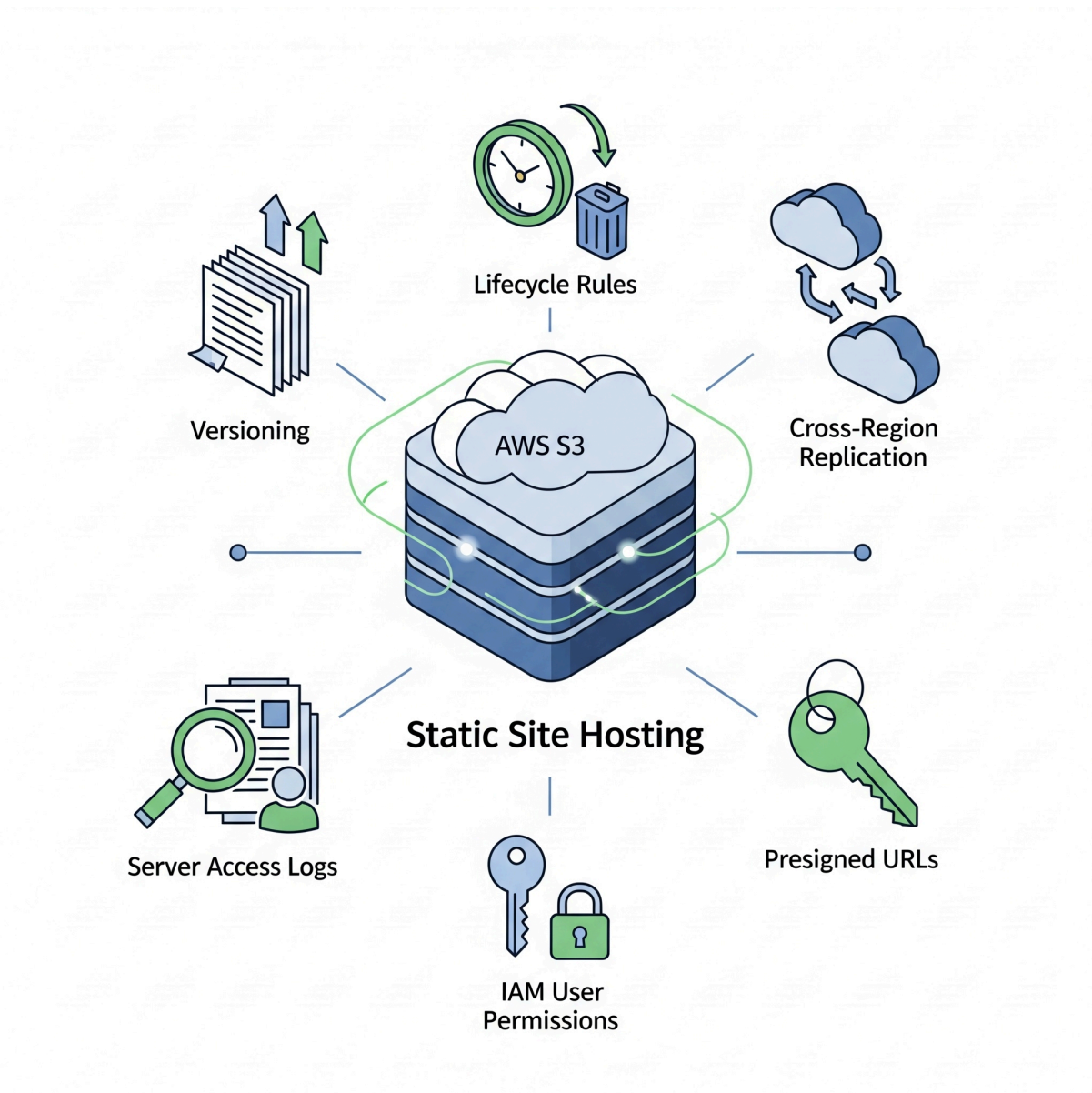
Project: End-to-End S3 Data & Website Management – Step-by-Step Guide

Goal: implement an industry-style S3 project that covers the important S3 features in a single practical mini-project:

- Website hosting (prod bucket)
 - Backups & replication (backup bucket)
 - Logging (logs bucket)
 - Versioning, lifecycle rules (cost optimization)
 - Encryption (SSE), bucket policies (least privilege)
 - IAM user with limited rights, presigned URLs
 - S3 Event → optional automation (Replication)
- Everything below is written so you can follow it exactly. Replace placeholder names with your real values.

Table of contents

1. Preparation — naming and AWS tips
2. Create buckets (prod, backup, logs)
3. Enable versioning & encryption
4. Static website hosting (prod) + public access policy (safe scope)
5. Lifecycle rules (versions + cost optimization)
6. Server access logging (logs bucket)
7. Backup automation options (Replication)
8. IAM user with limited upload rights (s3-uploader)
9. Pre-signed URL usage (temporary uploads or share)
10. Tests to validate (upload, rollback, restore)



1) Preparation — names, region, safety tips

- Choose region close to you (e.g., ap-south-1) to reduce latency and cost.
- Bucket naming convention (must be globally unique):
 - harshitha-website-prod-2025-<suffix>
 - harshitha-website-backup-2025-<suffix>
 - harshitha-website-logs-2025-<suffix>

- For safety during learning:
 - Keep objects small (no large video files).
 - Set lifecycle to delete older objects (so storage doesn't accumulate).

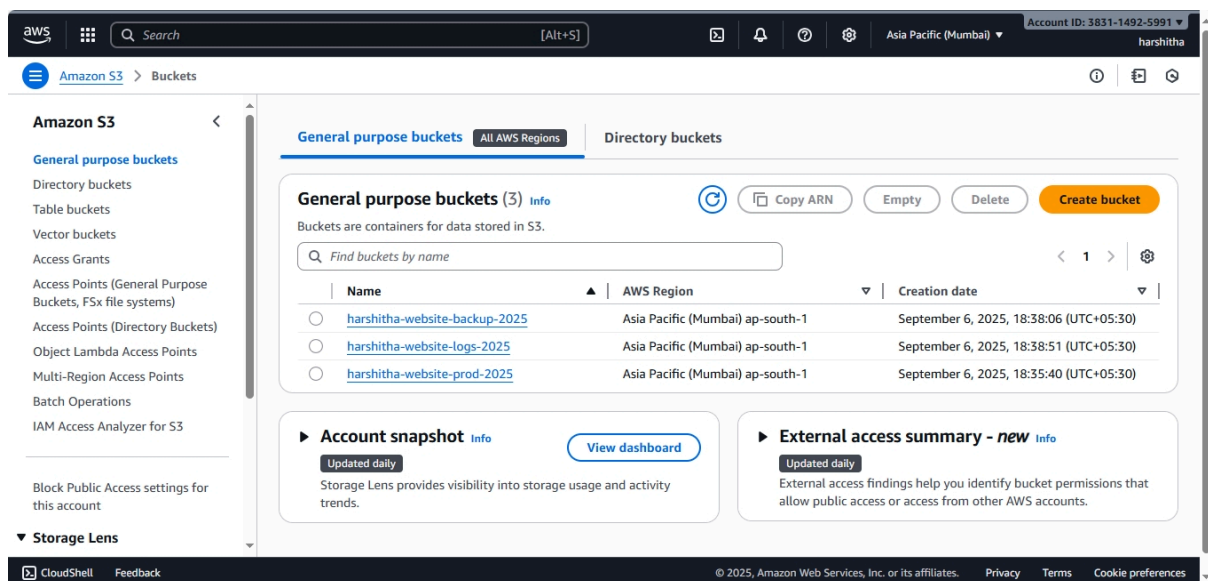
2) Create three buckets (Console steps + CLI)

Console steps (recommended for beginners)

1. AWS Console → S3 → Create bucket.
2. Enter bucket name (e.g., harshitha-website-prod-2025) and choose Region.
3. Uncheck “Block all public access” **only for the prod bucket** if you want static website public; leave **backup** and **logs** buckets with block public access ON.
4. Create the three buckets: prod, backup, logs.

CLI (optional)

```
aws s3api create-bucket --bucket harshitha-website-prod-2025 --region ap-south-1
--create-bucket-configuration LocationConstraint=ap-south-1
aws s3api create-bucket --bucket harshitha-website-backup-2025 --region ap-south-1
--create-bucket-configuration LocationConstraint=ap-south-1
aws s3api create-bucket --bucket harshitha-website-logs-2025 --region ap-south-1
--create-bucket-configuration LocationConstraint=ap-south-1
```



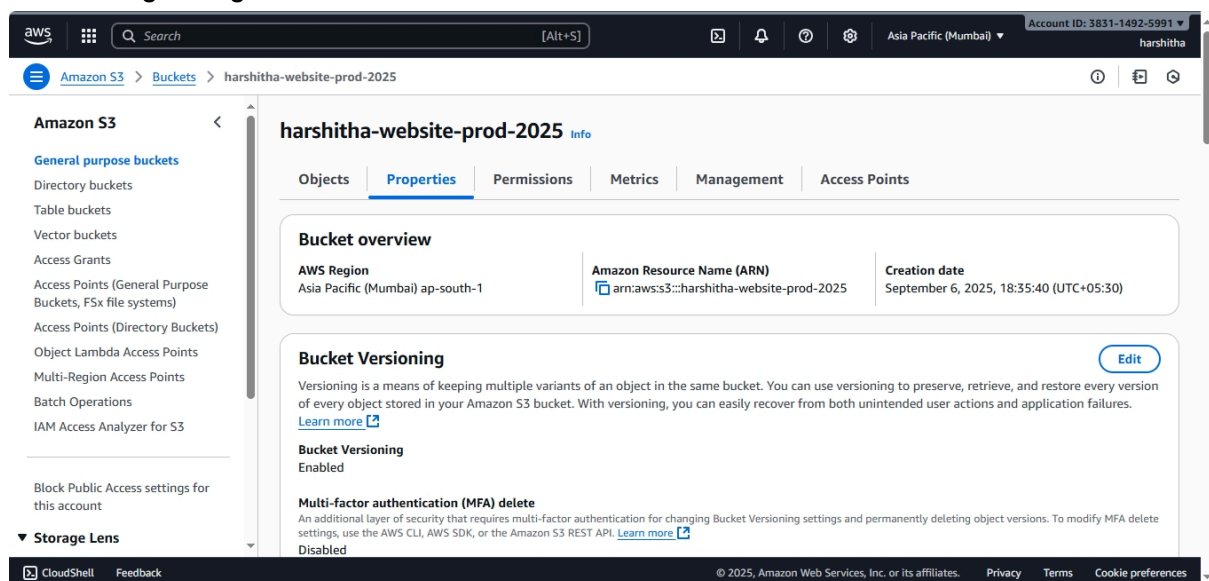
3) Enable Versioning & Encryption (Console + CLI)

3.1 Versioning (important for rollback)

Console: S3 → Select prod bucket → Properties → Versioning → Enable.

CLI:

```
aws s3api put-bucket-versioning --bucket harshitha-website-prod-2025
--versioning-configuration Status=Enabled
aws s3api put-bucket-versioning --bucket harshitha-website-backup-2025
--versioning-configuration Status=Enabled
aws s3api put-bucket-versioning --bucket harshitha-website-logs-2025
--versioning-configuration Status=Enabled
```



3.2 Server-Side Encryption (SSE-S3) — Amazon managed keys

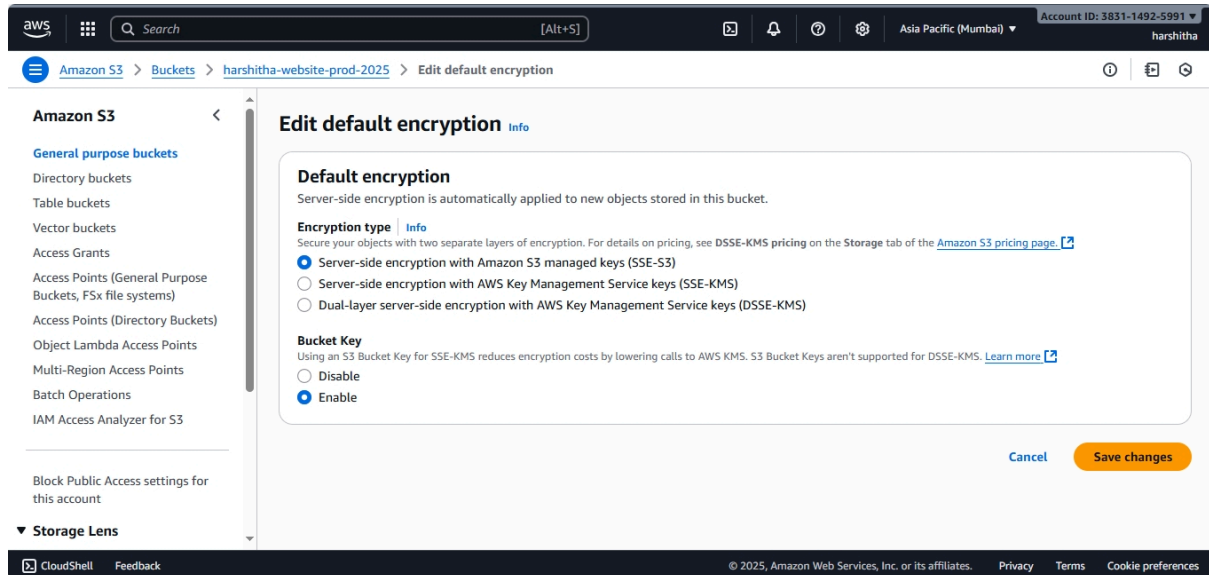
Console: S3 → bucket → Properties → Default encryption → Enable → Amazon S3-managed keys (SSE-S3).

CLI (example enabling default encryption):

```
aws s3api put-bucket-encryption --bucket harshitha-website-prod-2025
--server-side-encryption-configuration '{
  "Rules":[
    {"ApplyServerSideEncryptionByDefault":{"SSEAlgorithm":"AES256"}}
  ]
}'
```

Repeat for backup and logs buckets if you want them encrypted.

Note: SSE-KMS is possible for stricter control, but SSE-S3 is simpler for learning.



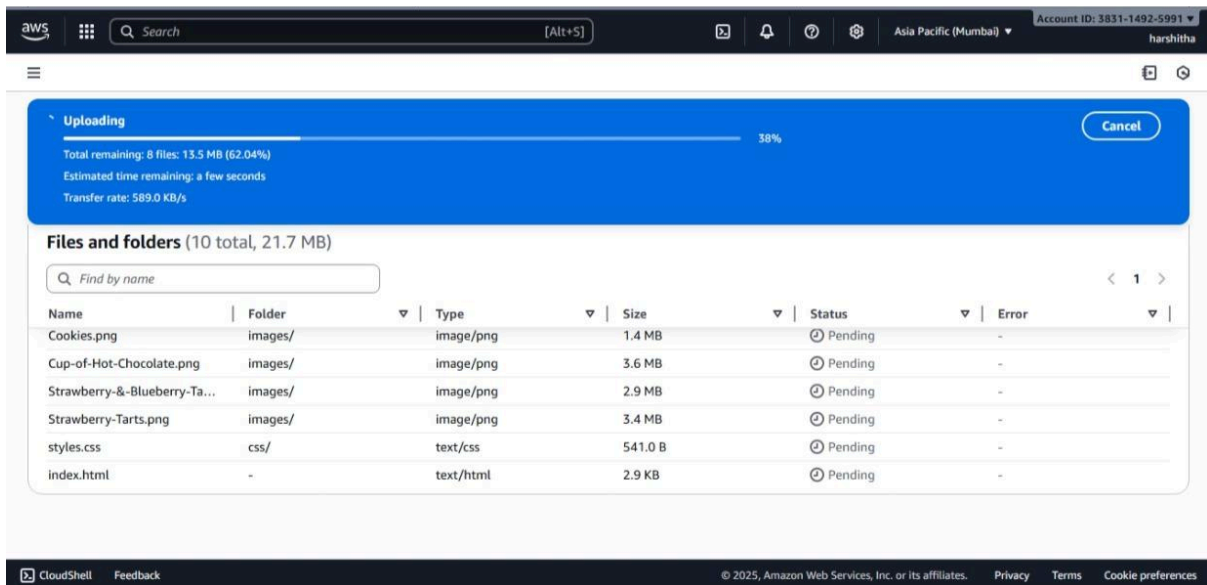
4) Static website hosting on prod bucket + safe public read

4.1 Upload website files

- Place your site files in a local folder `site/` (index.html, css, images).
- Console: S3 → prod bucket → Upload → add files & folders → Upload.

CLI:

```
aws s3 sync ./site/ s3://harshitha-website-prod-2025/ --acl public-read
```

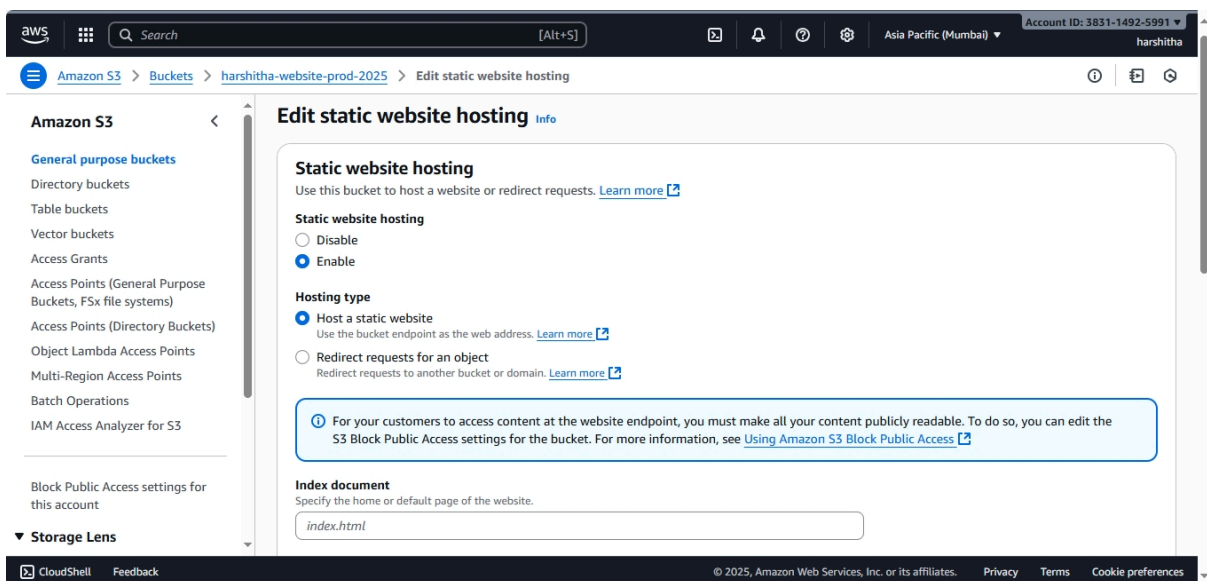


4.2 Enable Static Website Hosting (Console)

S3 → prod bucket → Properties → Static website hosting → Enable → Index document: `index.html` → Save.

Copy the Endpoint (example):

<http://harshitha-website-prod-2025.s3-website-ap-south-1.amazonaws.com>



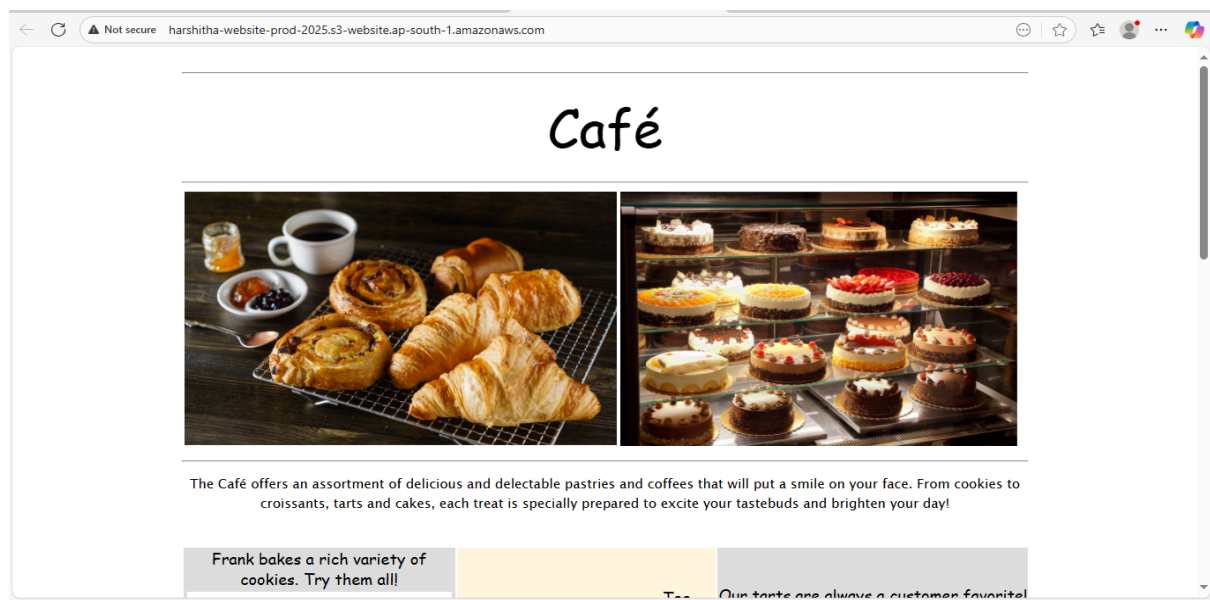
4.3 Bucket policy — public read *only for prod bucket objects*

If you want the prod site public, add this bucket policy (replace bucket name):

```
{
  "Version": "2025-09-06",
  "Statement": [
    {
      "Sid": "PublicReadForWebsite",
      "Effect": "Allow",
      "Principal": "*",
      "Action": ["s3:GetObject"],
      "Resource": ["arn:aws:s3:::harshitha-website-prod-2025/*"]
    }
  ]
}
```

Console → Permissions → Bucket policy → Paste JSON → Save.

Important: Do NOT apply this policy to backup or logs buckets.



5) Lifecycle rules (cost optimization / versions)

Goals

- Move non-current versions to Glacier/Archive after 30 days.
- Permanently delete old non-current versions after 365 days (example).
- Move backups to Infrequent Access (IA) after 60 days.

Lifecycle Rules Setup

Step 5A: Enable Versioning

1. Go to **S3 Console** → **your prod bucket**.
2. Open **Properties** tab.
3. Scroll to **Bucket Versioning** → **Edit** → **Enable** → **Save**.
 - This is mandatory for handling non-current versions.

Step 5B: Rule 1 → Non-current versions → Glacier

1. Go to **Management** → **Lifecycle rules** → **Create rule**.
2. Rule name: **NonCurrent-To-Glacier**.
3. Scope: Apply to **all objects**.
4. Choose action: **Transition noncurrent versions**.
 - After 30 days → **Glacier Flexible Retrieval** (or Deep Archive if you want cheapest).
5. Save rule.

Step 5C: Rule 2 → Expire non-current versions after 365 days

1. Again **Create rule**.
2. Rule name: Delete-Old-NonCurrent.
3. Scope: All objects.
4. Action: **Expire noncurrent versions**.
 - After 365 days.
5. Save rule.

Step 5D: Rule 3 → Move backups to Standard-IA after 60 days

1. Again **Create rule**.
2. Rule name: Backups-To-IA.
3. Scope: **Limit by prefix** → backups/.
 - (This means only objects under folder backups / are affected).
4. Action: **Transition current versions**.
 - After 60 days → **S3 Standard-IA**.
5. Save rule.

Verification

- In **Management** → **Lifecycle rules**, you should see **3 enabled rules**.
- For testing purpose, instead of 30/60/365 → set **1** or **2** days → wait 24 hrs → you will see transitions in **Storage Class** column of object.

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Account ID: 3831-1492-5991

harshitha

Amazon S3

Buckets

harshitha-website-prod-2025

harshitha-website-prod-2025

Objects

Properties

Permissions

Metrics

Management

Access Points

Lifecycle configuration

To manage your objects so that they are stored cost effectively throughout their lifecycle, configure their lifecycle. A lifecycle configuration is a set of rules that define actions that Amazon S3 applies to a group of objects. Lifecycle rules run once per day.

Default minimum object size for transitions
All storage classes 128K

Lifecycle rules (2)

View details

Edit

Delete

Actions

Create lifecycle rule

Use lifecycle rules to define actions you want Amazon S3 to take during an object's lifetime such as transitioning objects to another storage class, archiving them, or deleting them after a specified period of time. [Learn more](#)

	Lifecycle rule name	Status	Scope	Current version act...	Noncurrent version...	Expired object dele...	Incomplete multip...
☐	NonCurrent-To-Glacier	Enabled	Entire bucket	-	Transition to Glacier Flex	-	-
☐	Delete-Old-NonCurrent	Enabled	Entire bucket	-	Permanently delete	-	-

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Account ID: 3831-1492-5991

harshitha

Amazon S3

Buckets

harshitha-website-backup-2025

harshitha-website-backup-2025

Objects

Properties

Permissions

Metrics

Management

Access Points

Lifecycle configuration

To manage your objects so that they are stored cost effectively throughout their lifecycle, configure their lifecycle. A lifecycle configuration is a set of rules that define actions that Amazon S3 applies to a group of objects. Lifecycle rules run once per day.

Default minimum object size for transitions
All storage classes 128K

Lifecycle rules (1)

View details

Edit

Delete

Actions

Create lifecycle rule

Use lifecycle rules to define actions you want Amazon S3 to take during an object's lifetime such as transitioning objects to another storage class, archiving them, or deleting them after a specified period of time. [Learn more](#)

	Lifecycle rule name	Status	Scope	Current version act...	Noncurrent version...	Expired object dele...	Incomplete multip...
☐	Backups-To-IA	Enabled	Filtered	Transition to Standard-IA	-	-	-

[View lifecycle configuration](#)

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates.

Privacy

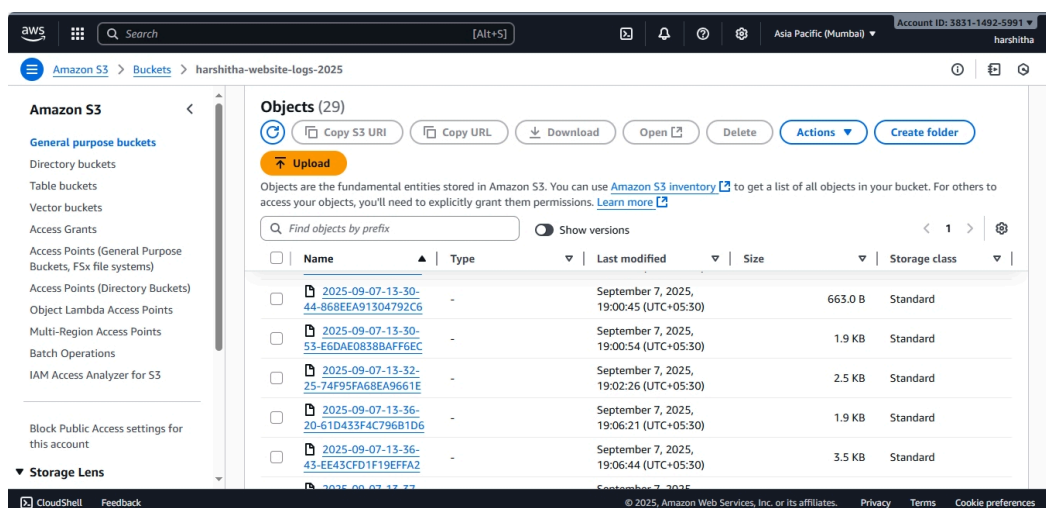
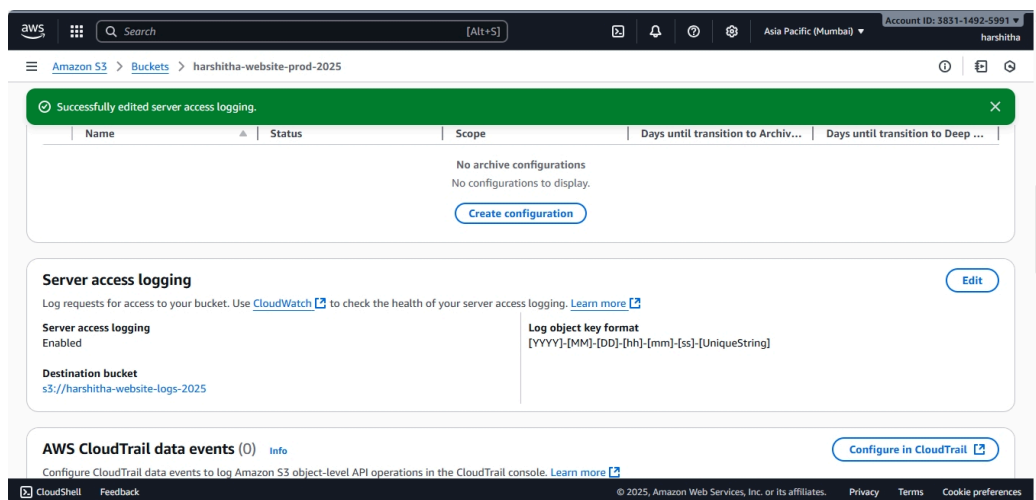
Terms

Cookie preferences

6) Server Access Logging (store S3 access logs into logs bucket)

Console steps:

1. Go to prod bucket → Properties → Server access logging → Edit.
2. Enable logging and select target bucket: harshitha-website-logs-2025.
3. Optional: set prefix prod-logs/.
4. Save.



Note: The logs bucket must allow S3 logging delivery; console will set required permissions automatically when you enable from UI.

7) Backup automation

Cross-Region Replication (CRR) — recommended for backups across regions

Requirements:

- Both source and destination buckets must have versioning enabled.
- A replication role/permissions are required (console can create this automatically).

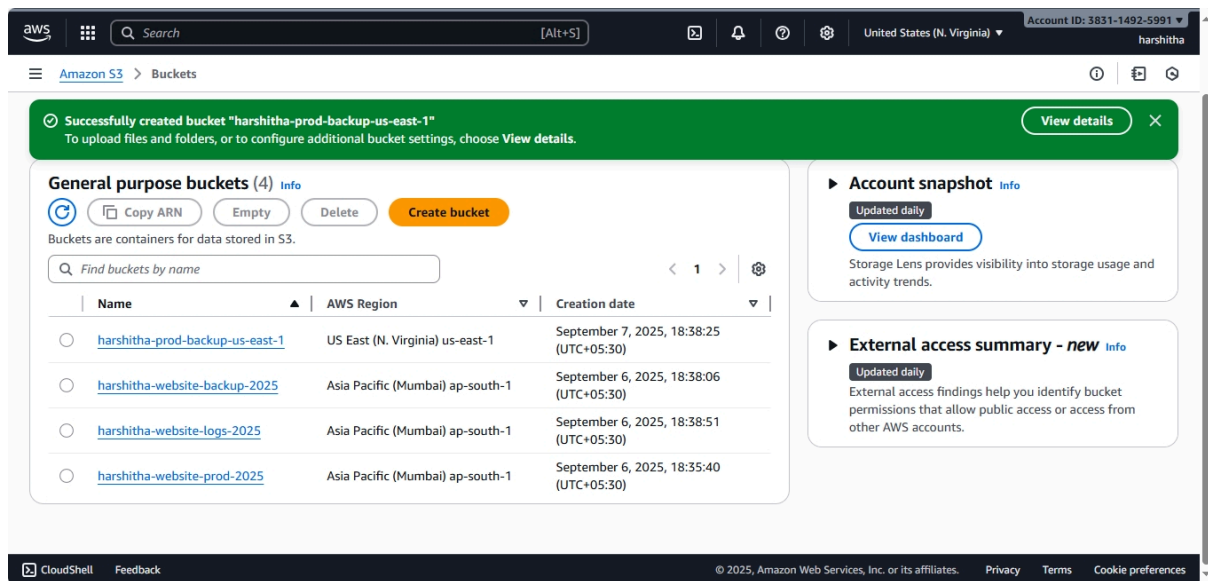
Steps to Enable Cross-Region Replication

1. Prerequisites

- **Both buckets must have Versioning enabled.**
 - Prod bucket (already enabled in Step 3).
 - Backup bucket (create new one with versioning ON).

2. Create Backup Bucket

- Go to **S3** → **Create bucket**.
- Name: harshitha-prod-backup-us-east-1.
- Region: select different region (like N. Virginia).
- Enable **Bucket Versioning**.
- Keep Block Public Access = ON.
- Click **Create bucket**.



3. Enable Replication on Prod Bucket

- Go to **prod bucket** → **Management** → **Replication rules** → **Create rule**.
- Rule name: replicate-to-backup.
- Status: Enabled.
- Choose **Apply to all objects**.
- Destination → select backup bucket (harshitha-prod-backup-us-east-1).
- IAM Role → let AWS create new role (e.g., s3-replication-role).
- Save rule.

aws Search [Alt+S] Asia Pacific (Mumbai) Account ID: 3831-1492-5991 harshitha

Amazon S3 > Buckets > harshitha-website-prod-2025 > Replication rules

Replication configuration successfully updated
If changes to the configuration aren't displayed, choose the refresh button. Changes apply only to new objects. To replicate existing objects with this configuration, choose **Create replication job**.

Replication rules (1) View details Edit rule Delete Actions Create replication rule

Use replication rules to define options you want Amazon S3 to apply during replication such as server-side encryption, replica ownership, transitioning replicas to another storage class, and more. [Learn more](#)

Replication rule name	Status	Destination bucket	Destination Region	Priority	Scope	Storage class	Replica owner	Replication Time Control	KMS-encrypted objects (SSE-KMS or DSSE-KMS)	Replication modification
replicate-to-backup	Enabled	s3://harshitha-prod-backup-us-east-1	US East (N. Virginia) us-east-1	0	Entire bucket	Same as source	Same as source	Disabled	Do not replicate	Disabled

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

4. Test Replication

- Upload a new file in prod bucket.

aws Search [Alt+S] Asia Pacific (Mumbai) Account ID: 3831-1492-5991 harshitha

Upload succeeded
For more information, see the **Files and folders** table.

Summary

Destination	Succeeded	Failed
s3://harshitha-website-prod-2025	1 file, 0 B (0%)	0 files, 0 B (0%)

Files and folders Configuration

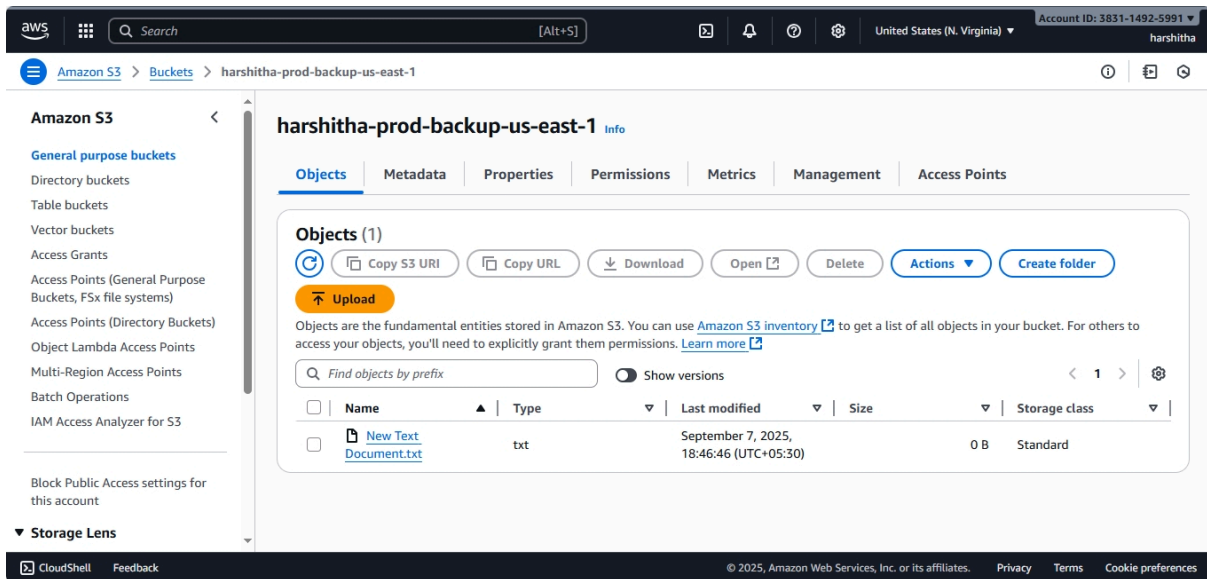
Files and folders (1 total, 0 B)

Find by name

Name	Folder	Type	Size	Status	Error
New Text Document.txt	-	text/plain	0 B	Succeeded	-

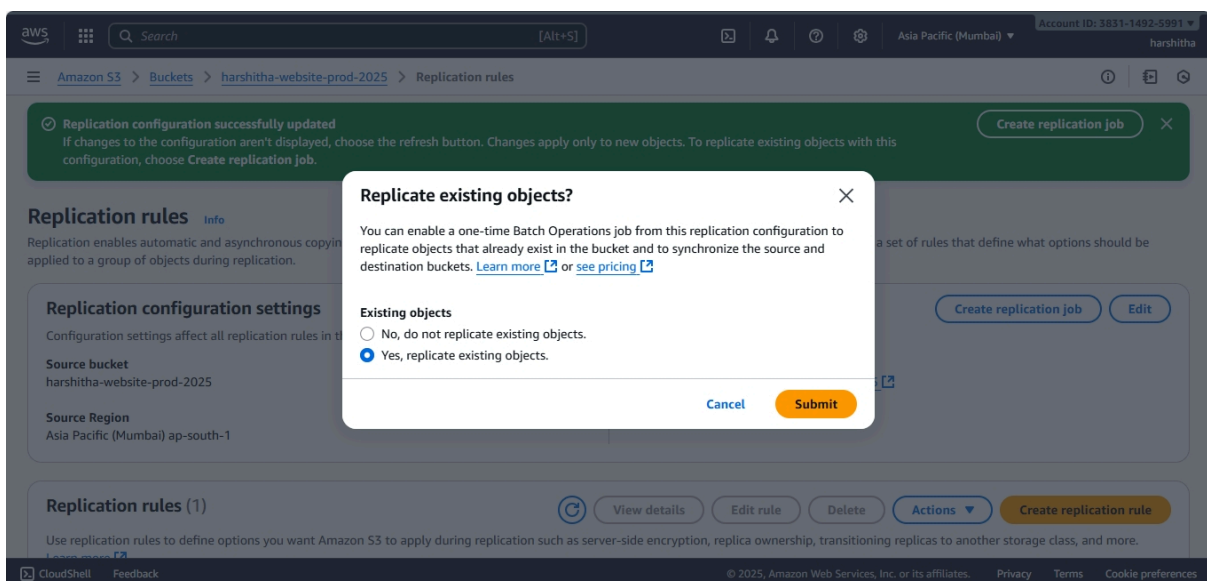
CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

- Wait few minutes → check backup bucket in other region.
- File should automatically appear.



Verification & Notes

- Replication works only for **new objects** after enabling CRR (old objects won't move).
- If you want old objects too → select **Replicate existing objects** while creating rule.



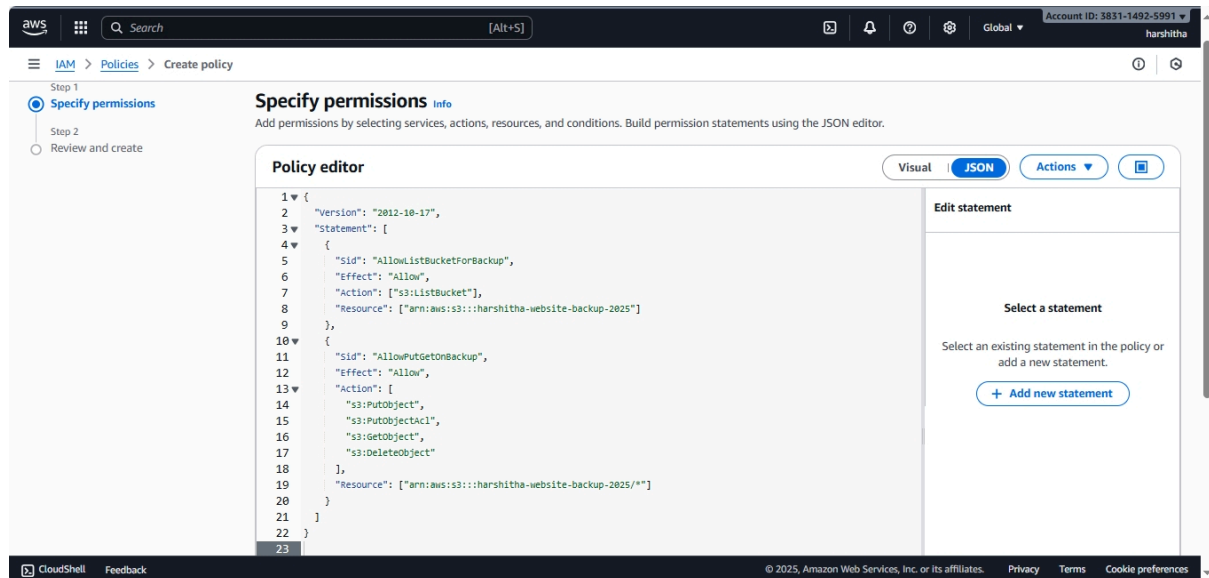
- Replication latency depends on AWS (usually within seconds–minutes).

This ensures any new object uploaded (or new version) is replicated to **backup** automatically.

8) Create IAM user s3-uploader with limited permissions

Create an IAM user that can only upload to backup bucket (useful for automation or other persons).

Policy JSON (s3-uploader-policy.json)



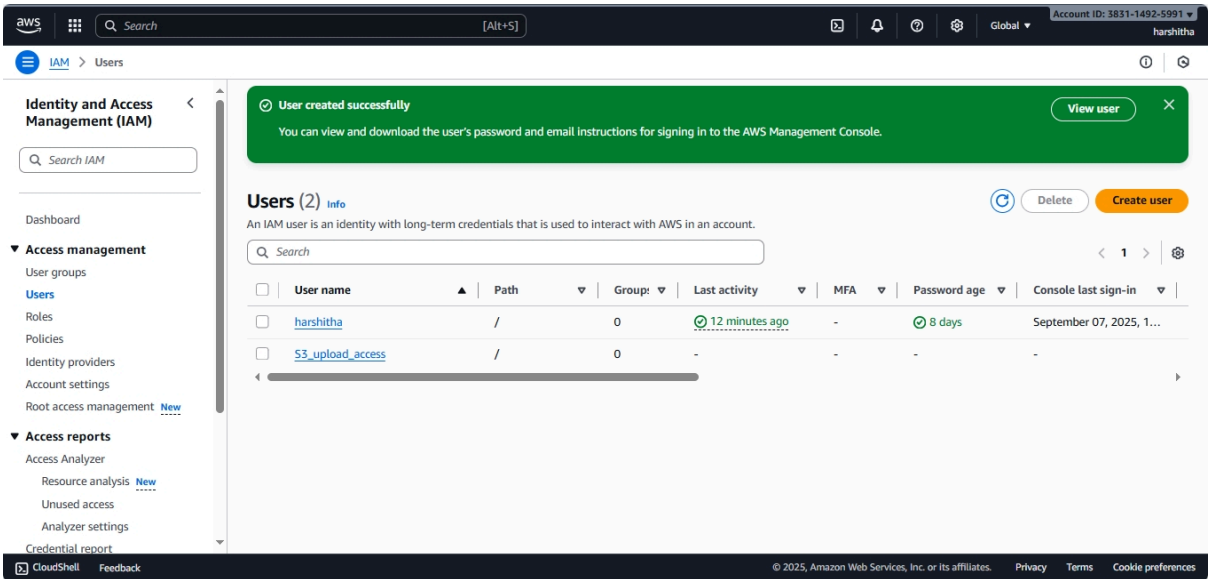
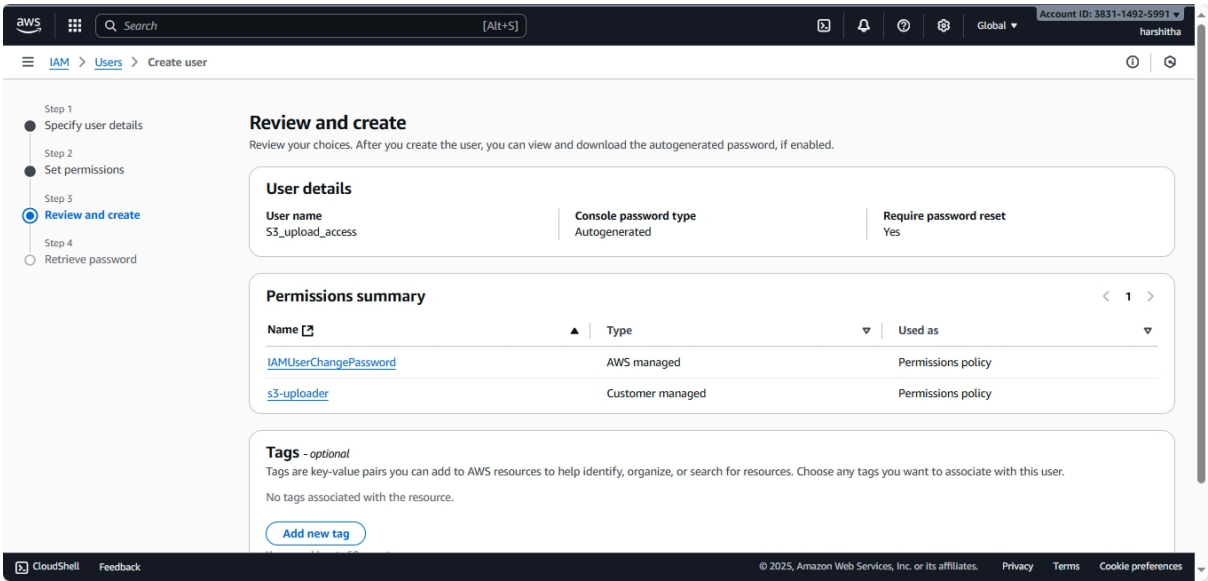
Console steps

- IAM → Users → Add user **s3-uploader** → Programmatic access (checkbox) → Attach policy → Create inline policy using JSON above → Create user and save access key (or use AWS CLI to create).

CLI (attach policy inline):

```
aws iam create-user --user-name s3-uploader
aws iam put-user-policy --user-name s3-uploader --policy-name S3UploaderPolicy
--policy-document file:///s3-uploader-policy.json
aws iam create-access-key --user-name s3-uploader
```

Store the keys securely or rotate them frequently.



9) Pre-signed URL for secure temporary upload/download

Pre-signed URLs are useful to give temporary write access without creating IAM users.

Generate presigned PUT URL (CLI / AWS CLI v2 or Python):

CLI (using `aws s3 presign` only creates GET presigned URLs). For PUT you can use AWS SDK or `aws cli` via `aws s3api put-object` is not `presign`. Use Python `boto3`:

Python example to generate presigned PUT URL:

```
import boto3

s3 = boto3.client('s3')
url = s3.generate_presigned_url(
    ClientMethod='put_object',
    Params={'Bucket': 'harshitha-website-backup-2025', 'Key': 'uploads/file1.txt'},
    ExpiresIn=3600
)
print(url)
```

```
curl --upload-file ./localfile.txt "PASTE_URL_HERE"
```

A user can use HTTP PUT to upload to that URL for one hour.

GET presigned URL (CLI):

```
aws s3 presign s3://harshitha-website-backup-2025/uploads/file1.txt --expires-in 3600
```

Note: Link will be generated like this,you can access it for one hour

```
https://harshitha-website-backup-2025.s3.ap-south-1.amazonaws.com/uploads/file1.txt?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=AKIAVSM3N26T5VPLQZ7W%2F20250907%2Fap-south-1%2Fs3%2Faws4_request&X-Amz-Date=20250907T132919Z&X-Amz-Expires=3600&X-Amz-SignedHeaders=host&X-Amz-Signature=2e3530178b27e6fa521943c6779dbfdc2e13e7b67534430441d7c50bcaba765a
```

10) Tests & validation (follow these in order)

1. Upload a test file to prod

- Upload `index.html` and `test.txt`.
- Confirm `index.html` loads from S3 website endpoint.

2. Create a second version

- Edit `index.html` locally and upload again (same key).
- In S3 console → prod bucket → show versions → confirm multiple versions exist.

3. Rollback to previous version

- In S3 console, copy an older version as current (download old version and re-upload, or use `aws s3api get-object --version-id ...`).
- Verify website content reflects older version.

4. Replication / Backup test

- CRR: check backup bucket for replicated object and versions.

5. Lifecycle test (non-current transition)

- You cannot fast-forward time; instead, simulate by adjusting lifecycle to a small number of days for test (e.g., `NoncurrentDays = 1`) then wait 24–48h. For production, restore the intended values.

6. Server Access logs

- Upload and access site; go to logs bucket and check for new log files (may take some time to appear).

7. Presigned URL test

- Generate presigned URL and attempt to upload via curl or browser PUT to confirm it works.

8. Security check

- Confirm backup and logs buckets are not public (Block Public Access ON).

- Confirm prod has only the intended public read policy.

Cleanup steps (to avoid charges)

1. Empty buckets and delete buckets:

```
aws s3 rm s3://harshitha-website-prod-2025 --recursive
```

```
aws s3 rb s3://harshitha-website-prod-2025
```

2. Delete IAM users / access keys created for this project.
3. Remove replication rules and disable versioning if you want (versioning cannot be disabled if there are delete markers; you can stop versioning but objects remain).

Helpful CLI snippets (cheat sheet)

Create bucket:

```
aws s3api create-bucket --bucket harshitha-website-prod-2025 --region ap-south-1  
--create-bucket-configuration LocationConstraint=ap-south-1
```

Enable versioning:

```
aws s3api put-bucket-versioning --bucket harshitha-website-prod-2025  
--versioning-configuration Status=Enabled
```

Put bucket policy:

```
aws s3api put-bucket-policy --bucket harshitha-website-prod-2025 --policy  
file://policies/bucket-policy-prod.json
```

Upload files:

```
aws s3 sync ./site/ s3://harshitha-website-prod-2025/ --acl public-read
```

Generate presigned GET:

```
aws s3 presign s3://harshitha-website-backup-2025/uploads/sample.txt --expires-in 3600
```

Create IAM user & inline policy (example):

```
aws iam create-user --user-name s3-uploader
aws iam put-user-policy --user-name s3-uploader --policy-name S3UploaderPolicy
--policy-document file:///policies/s3-uploader-policy.json
aws iam create-access-key --user-name s3-uploader
```

Delete (cleanup):

```
aws s3 rm s3://harshitha-website-prod-2025 --recursive
aws s3 rb s3://harshitha-website-prod-2025
```

Final notes – best practice reminders

- Use a separate backup bucket and keep it private. Use cross-region replication for resilience.
- Use lifecycle rules to avoid accumulating storage and costs.
- Keep sensitive logs and backup buckets private.
- Use versioning to enable rollback and auditing.
- Replace **SSE-S3** with **SSE-KMS** if you need more control / audit trails.
- For production, prefer CloudFront + OAC and keep bucket private (we'll do that later).